智能线激光三维测量系统

——基于 PYNQ-Z2 的 FPGA 流式激光条纹处理与 360°尺寸检测

摘要

线激光三维测量是零件尺寸检测、逆向建模和质量控制中的基础手段，对小型零件的在线抽检尤为重要。然而，现有线激光测量系统大多把条纹提取、三维重建和尺寸计算交给上位机完成，图像需要整帧传输，系统体积大、时延长；已有的 FPGA 方案也多只加速条纹中心提取，几何解算和尺寸判定仍依赖电脑；此外，物体反光和镜头失焦会使条纹质量起伏不定，造成漏点和误点，扫描过程需要人工值守。针对这些问题，本作品基于 PYNQ-Z2 设计了一套嵌入式智能线激光三维测量系统。首先，在可编程逻辑中构建了从像素流到三维坐标的全流水处理链，把基于 Hessian 矩阵的亚像素条纹中心提取、跨行关联和定点几何解算串联起来，使每一帧图像都在片上完成重建；其次，把高度统计、截面台阶测量和点云体素渲染也放进可编程逻辑，尺寸结果和点云直接在 HDMI 画面上实时呈现，不再需要上位机；最后，建立了由条纹质量分级、激光自动曝光和无效视角重采组成的采集闭环，并配合“步进—停稳—采集”的转台同步，实现一键完成 360°扫描。实测表明，系统能随相机帧率逐帧处理图像（约 26.5 帧/s），条纹中心误差的中位数约 0.02 像素，单帧处理时间不到同一算法 PC 端参考实现的三分之一；一圈扫描约 2.5 min，可获得约 6.6 万个测量点，阶梯圆柱和花瓶的直径与卡尺读数相差不超过 0.32 mm；整个设计占用 XC7Z020 约 73.5%的查找表资源，片上功耗约 2.37 W。

第一部分　作品概述

1.1 功能与特性

系统围绕“扫得出、量得准、看得见”设计，主要功能如下（测量原理见图 1）：

（1）360°旋转扫描：BTN0 一键启动，按“步进—停稳—采集”完成 200 个视角，输出点云与封闭 STL 网格；

（2）多截面尺寸测量与超差判定：按高度分层拟合直径，生成直径—高度曲线并与公差比较；

（3）定点轮廓实时测量：在激光截面上测量台阶高差和间隙，统计由 PL 硬件完成；

（4）采集质量评估与自适应：按条纹强度、宽度、饱和与完整性分级，PL 自动调节曝光，无效视角自动重采；

（5）实时可视化：三页 HDMI 界面分别显示相机画面与 PL 点云、截面测量、扫描进度与直径曲线；

（6）开机即用：配置固化在 SD 卡启动分区，上电自动加载固件与演示程序。

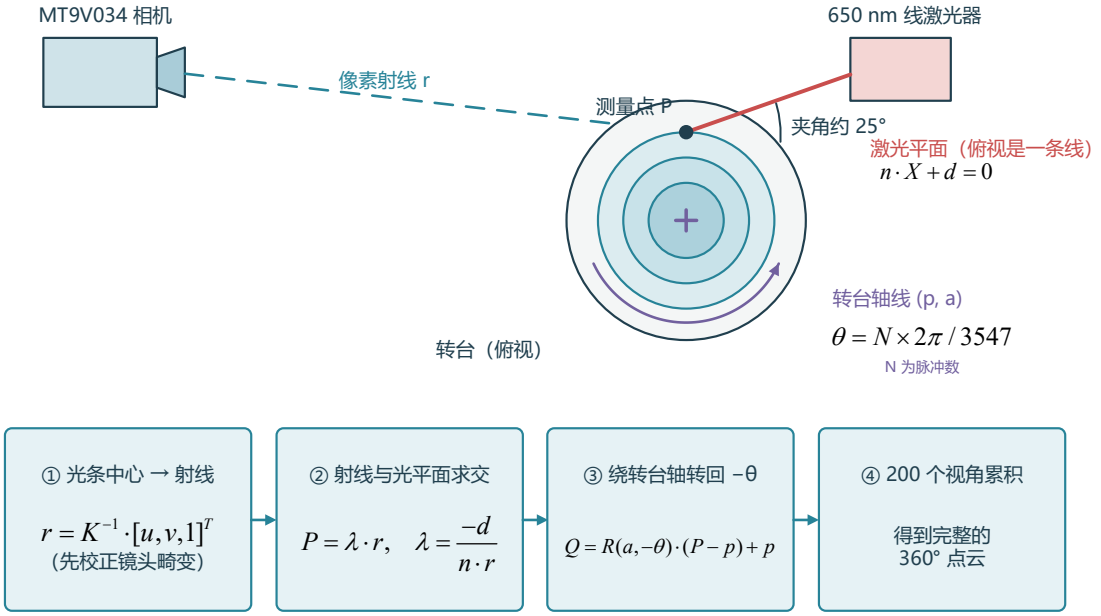


图 1　激光三角测量与转台扫描原理

1.2 应用领域

本系统适用于轴、套、阶梯轴等小型回转体零件的尺寸抽检，3D 打印件的尺寸复核与形貌存档，以及产线工位上台阶高差、间隙等定点轮廓检测。与接触式量具相比，一次扫描即可得到全周轮廓和各截面直径；与依赖 PC 的商用扫描仪相比，整套处理在

一块低成本开发板上完成，不需要上位机，便于嵌入设备。系统同时可作为激光三角测量、相机标定与 FPGA 图像流处理的教学实验平台，光条中心、截面和点云等中间结果都可以单独显示和导出。高反光、透明和深槽物体会造成条纹缺失或多峰，属于需要额外处理的工况。

1.3 主要技术特点

（1）全流水 PL 处理：从相机像素到三维坐标，再到点云显示，全部在 FPGA 里连续流水完成，ARM 不处理像素；两路求解并行，某一行候选点太多时只丢弃这一行，不影响整帧。

（2）定点数几何：用整数表示小数——像素坐标带 12 位二进制小数，毫米坐标带 16 位二进制小数；乘法用硬件乘法器，除法用移位相减除法器，结果与浮点参考程序逐项核对一致。

（3）质量分级与自适应：每个点分为“预览级”和“测量级”；FPGA 根据条纹亮度自动调节曝光；条纹变宽时可切换为水平 2 合 1 模式。

（4）完整标定链：依次标定相机内参、激光平面、转台轴线和转角系数，再用阶梯样件微调，最后用卡尺检验。

（5）可靠性设计：每个数据包都校验指纹和序号；电机由“租约看门狗”保护，程序停止续租就自动断电；开机时先核对文件校验值再加载固件。

1.4 主要性能指标

表 1　主要性能指标（板上实测）

指标	实测结果
相机输入	752×480，8 bit 灰度，PCLK 24 MHz
PL 处理能力	逐帧处理，约 26.5 帧/s（随相机帧率），0 错误包
中心提取精度	合成真值条纹：误差中位数 0.02 像素，P95 0.05 像素
单次扫描	200 视角/圈，约 2.5 min；测量级视角 195～199 个，跳过 0 个
尺寸精度（卡尺真值）	阶梯圆柱直径−0.32～−0.10 mm（复核）；花瓶颈部−0.27～+0.05 mm（独立）
资源占用	LUT 73.5%，FF 47.8%，BRAM 78.9%，DSP 60.9%
时序与功耗	WNS +0.230 ns，WHS +0.011 ns，片上功耗 2.37 W

1.5 主要创新点

① 从相机像素到三维坐标、再到点云显示，全程在 FPGA 内流水完成，ARM 不处理像素；

② 高度、半径和台阶间隙的统计直接由 FPGA 完成，ARM 只需读取结果；

③ 条纹质量评估、自动曝光和无效视角重采组成闭环，扫描质量不靠人工盯着；

④ 标定与验证分开：用棋盘标定几何参数，用阶梯样件微调，再用另一件独立打印的样件和卡尺检验；

⑤ 开机即用，一键扫描，电机有看门狗保护。

1.6 设计流程

先用 Python 建立条纹提取与几何解算的参考模型，再逐模块编写 RTL，在仿真中与参考模型逐字段比对；综合实现后检查时序与跨时钟域，再上板联调相机、DMA 和 HDMI；随后完成标定，扫描实物并以卡尺验证，最后固化到启动分区（图 2）。

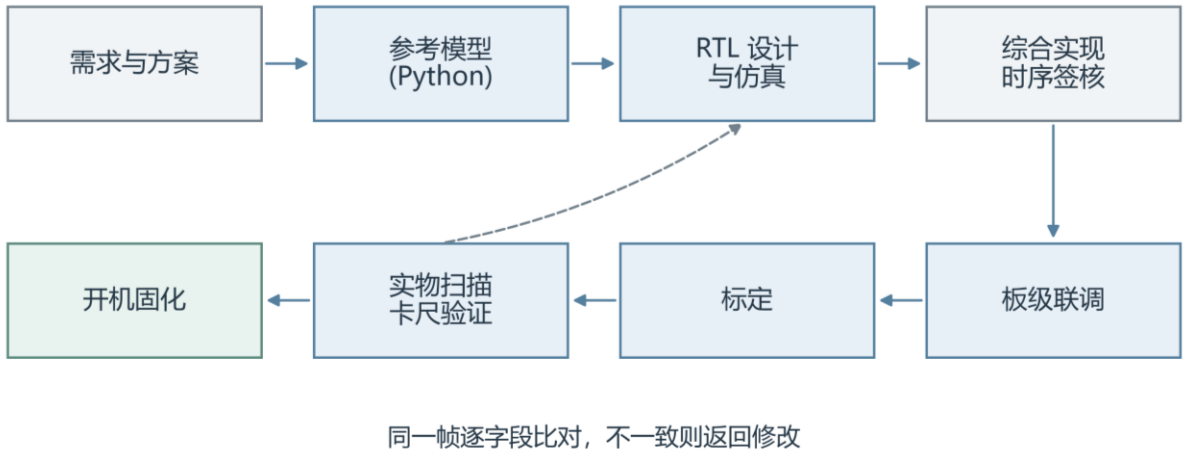


图 2　设计流程

第二部分　系统组成及功能说明

2.1 整体介绍

系统总体结构如图 3 所示。相机通过 8 位并行接口把 752×480 的灰度图像逐像素送入 FPGA。FPGA 内部依次完成采集与时钟转换、条纹中心提取和几何解算，得到每个条纹点的三维坐标。这些坐标同时用于三件事：打包后经 DMA 存入内存，供 ARM 拼接点云；送入测量统计模块，直接算出高度、半径和台阶；送入渲染模块，实时画成

点云叠加到 HDMI 画面上。ARM 通过 AXI 总线读取数据包、下发转台步数，并读写各模块的寄存器。

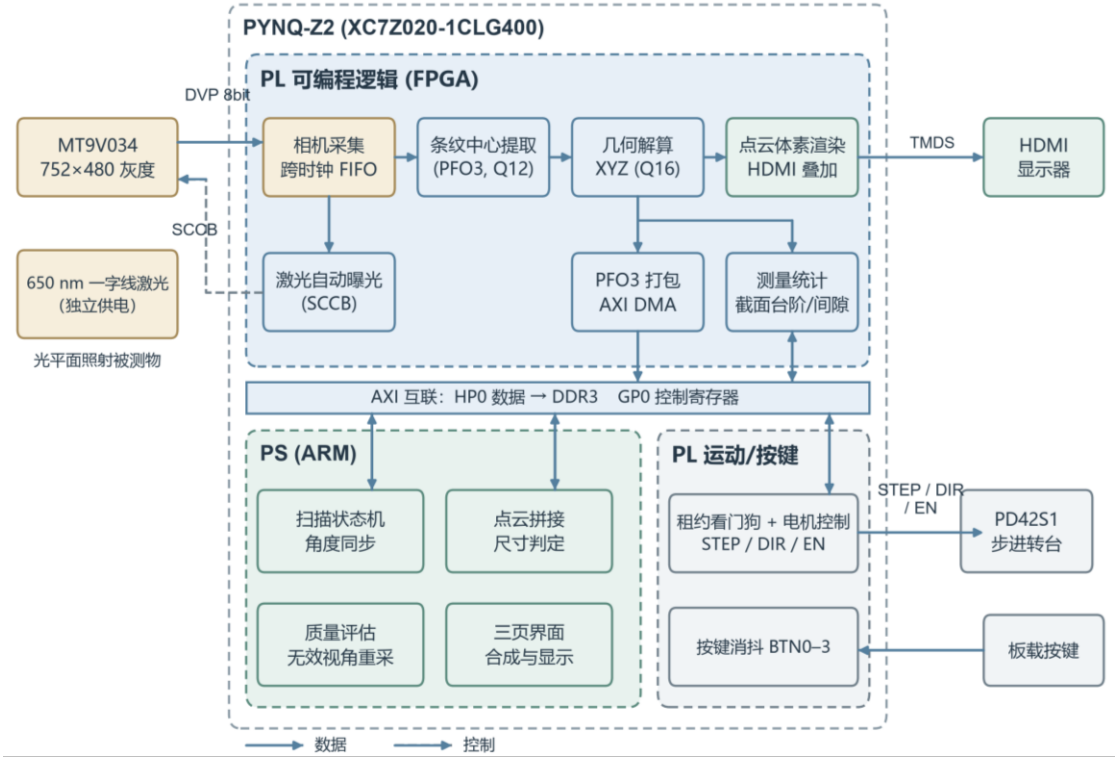


图 3　系统总体框图

软硬件的分工如图 4 所示：逐像素、逐点、时序固定的处理放在 FPGA，保证每一帧都能按相机帧率处理完；流程控制、点云拼接、尺寸判定和界面分支多、数据量小，放在 ARM。两边只通过数据包和寄存器交换信息，所以界面卡顿不会影响采集。

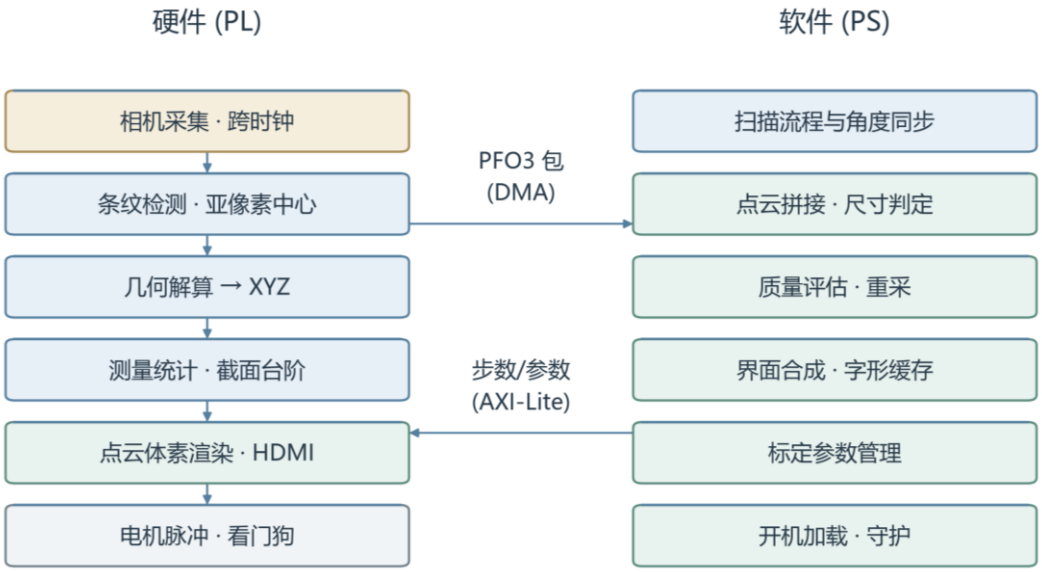


图 4　软硬件划分

2.2 硬件系统介绍

2.2.1 硬件整体介绍

硬件组成见表 2。转台由 PD42S1 闭环步进驱动器驱动，12 V 独立供电；FPGA 直接输出 STEP、DIR、EN 三路信号，转速 5 RPM，每个视角转 18 个脉冲。实测转台转过的角度只有指令角度的 0.902 倍，所以一圈折算为 3547 个脉冲。相机光轴与激光平面夹角约 25°，转台轴线距相机约 266 mm，在这个距离上一个像素约对应 0.26 mm。

表 2　硬件组成

模块	型号/参数	作用
主控	PYNQ-Z2，XC7Z020-1CLG400，512 MB DDR3	PL 流式处理，PS 控制与界面
相机	ATK-MT9V034，全局快门，752×480，8 bit	采集激光条纹图像
激光器	650 nm 一字线激光，独立供电	在物体表面投射光平面
转台	PD42S1 闭环步进驱动，脉冲模式，12 V	带动物体旋转，3547 脉冲/圈
显示	HDMI 显示器，1280×720	三页实时界面
交互	板载按键 BTN0～BTN3	启动/停止扫描、切换页面
存储	Micro SD 卡（PYNQ 镜像）	程序、标定参数、扫描数据



图 5　MT9V034 相机模块

2.2.2 机械设计介绍

机械部分包括相机安装、激光器支架和转台三部分（图 6）。相机模块通过自制的直插转接板直接插在 PYNQ-Z2 上，开发板放在支撑盒上以抬高视角；650 nm 线激光器装在带底座的万向支架上，可以调整光平面的方向，并单独供电；转台由 PD42S1 步进电机驱动，被测物放在转台中心附近。标定完成后，相机、激光器和转台的相对位置保持

不动。最终展示用三件样件：台阶块用来演示台阶高差和间隙测量，相邻台阶高差为 2、3、5 mm；阶梯圆柱是三段同轴圆柱，下、中、上段直径分别为 40、30、20 mm（设计每段高 16 mm、总高 48 mm），用来检验直径测量；波纹花瓶表面有扭转波纹，用来展示曲面细节和全周点云。台阶高差和阶梯圆柱直径都用游标卡尺核对过。阶梯圆柱在调试阶段参与过标定微调，所以它的复测结果只能作复核，不算独立精度；花瓶没有参与标定。宇航员模型只用于前期探索，不作为最终展示件。

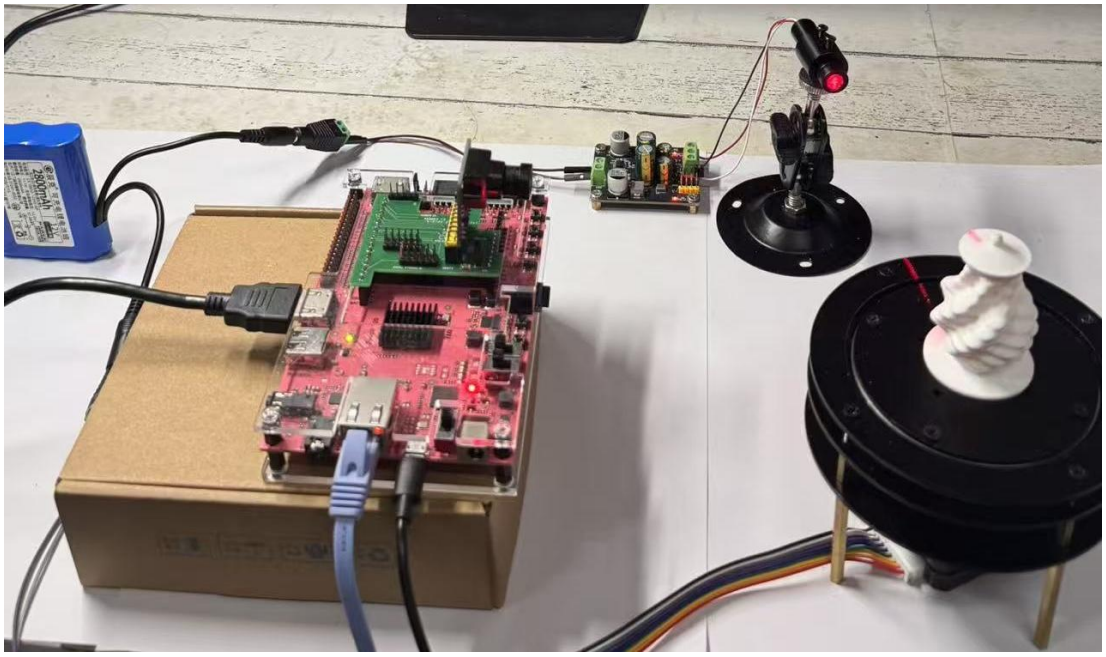


图 6　整机机械布局：左为 PYNQ-Z2 与相机，右上为线激光器支架，右下为转台与花瓶

2.2.3 电路各模块介绍

电路连接如图 7 所示（引脚编号取自实现后的约束文件）：相机的 8 位数据线、行/帧有效、复位、待机和 SCCB 接在扩展接口，像素时钟 PCLK 单独接到 PMOD A 的时钟引脚；转台的 STEP、DIR、EN 接 PMOD B；HDMI 由 FPGA 直接输出。电机使用独立的 12 V 电源。

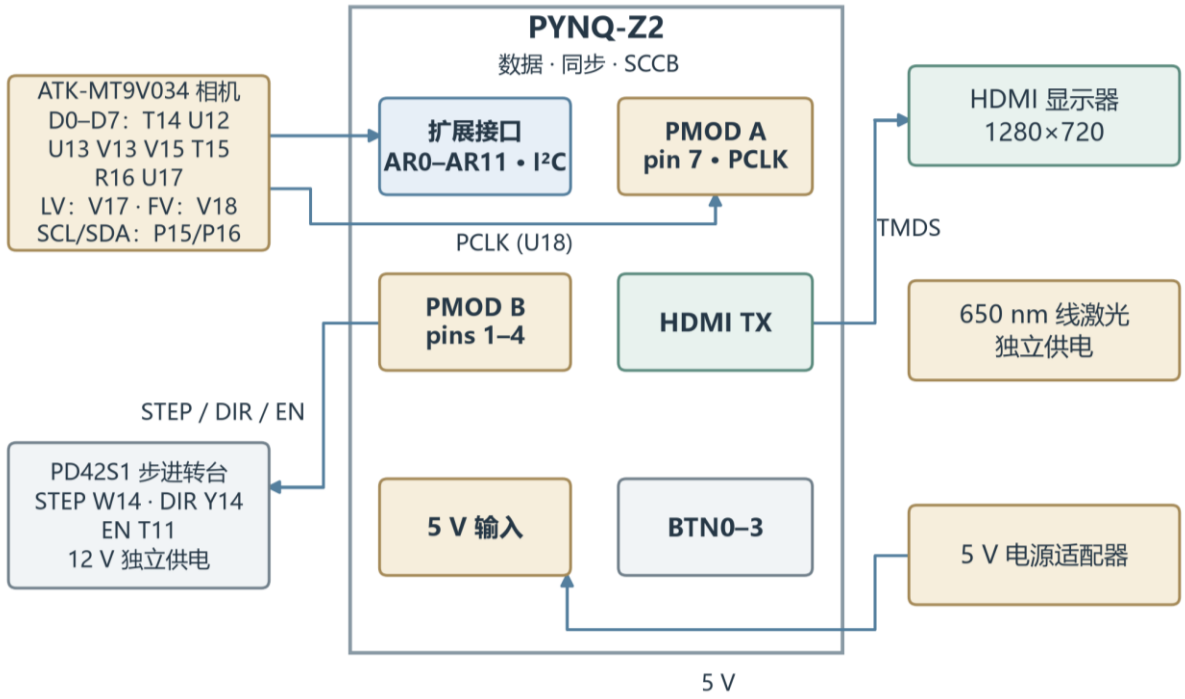


图 7　硬件连接示意图

为了少用飞线，另外设计了一块相机直插转接板（约 57×52 mm，图 8），带相机插座、主板接口、相机时钟和电机接口。该板已完成电气检查并装配使用，相机通过它直接插在 PYNQ-Z2 上（实物见图 23）。

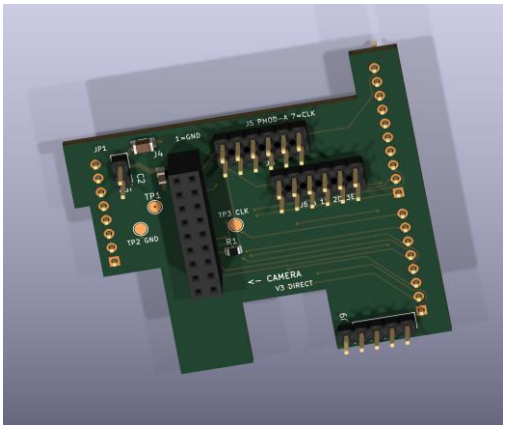


图 8　相机直插连接板设计（三维视图）

FPGA 内部按时钟分为三个区域（图 9）：相机时钟域（24 MHz）完成帧/行检测、像素打包、信号诊断和曝光决策；处理时钟域（100 MHz）包含包控制、条纹提取、两路求解、几何解算、测量统计、截面测量、点云视图、DMA、电机控制和按键；视频时钟域（125 MHz）负责 HDMI 输出。不同时钟之间只通过异步 FIFO、xpm_cdc 和双时钟 BRAM 传递数据，实现后跨时钟域严重项为 0。

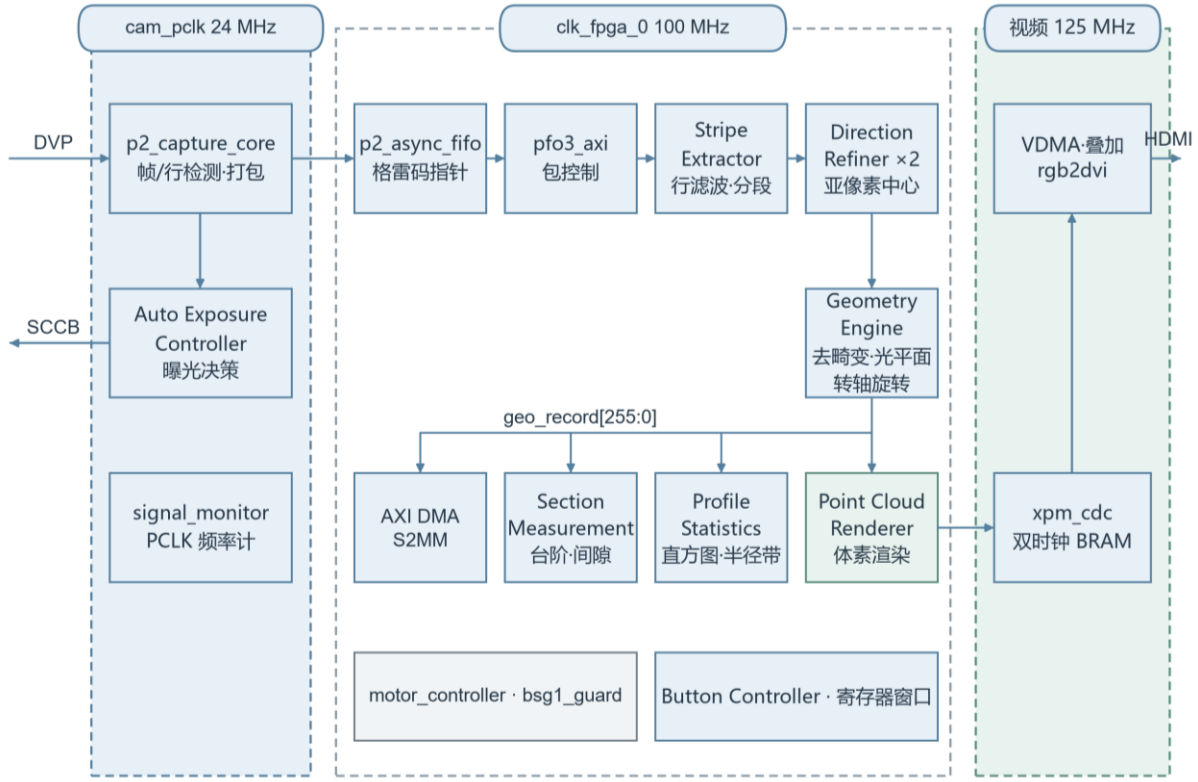


图 9　PL 逻辑结构与时钟域

相机数据跨时钟用的是格雷码指针异步 FIFO。图 10 是它的仿真波形：写时钟 24 MHz，连续写入 6 个像素字；读侧 100 MHz，按顺序读出，数据一致，没有溢出。

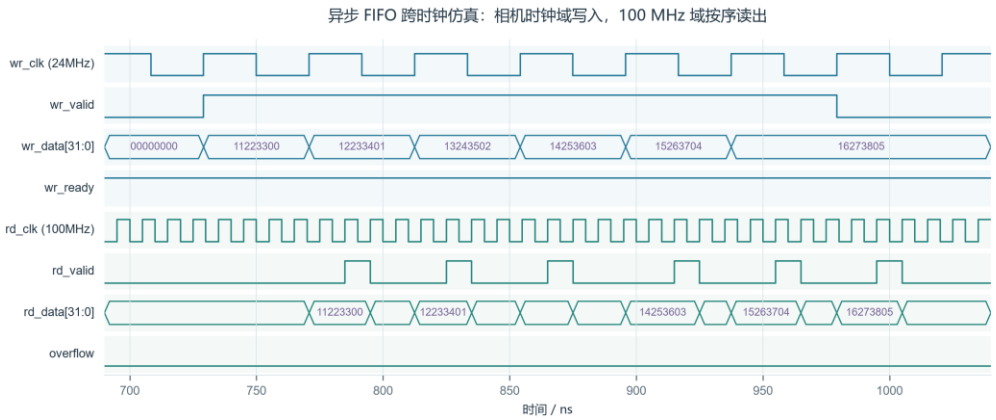


图 10　异步 FIFO 跨时钟仿真波形

PS 通过 AXI-Lite 访问各模块寄存器，地址分配见表 3。

表 3　PL 寄存器地址分配

模块	基地址/窗口	内容
PFO3 包控制	0x4000_0000 + 0x000	启动/状态、帧计数、错误码
直通流控制	+ 0x024～0x034	直通使能、快照请求、包计数
帧参数快照	+ 0x100～0x1FF	ROI、阈值、算法版本、几何参数
点云体素视图	+ 0x400～0x4FF	视角、体素窗口、渲染状态
实时测量统计	+ 0x500～0x5FF	Z 最小/最大/中位数、半径带
按键	+ 0x600～0x6FF	消抖状态、事件计数
截面测量	+ 0x700～0x7FF	台阶高差、间隙、噪声
PFO3 DMA	0x4043_0000	S2MM/MM2S 描述符
电机控制器	0x43C4_0000	步数、周期、方向、状态
租约看门狗	0x43C5_0000	续租、使能、故障码

2.3 软件系统介绍

2.3.1 软件整体介绍

软件分为启动层、主进程和工具三层（图 11）。开机时，启动脚本 boot.py 先核对比特流文件的校验值，再加载 FPGA 固件并启动演示服务；演示服务每轮运行 30 分钟，出错会自动重启，并监控磁盘空间。系统主进程用多个线程分工：一个线程控制相机和 FPGA、接收数据包；一个解码三维坐标并评估质量；一个运行扫描状态机；一个让 FPGA 里的点云随转台角度旋转；一个合成 HDMI 界面。数据写盘放在后台队列，扫描结束后统一校验。电脑端工具负责标定拟合、点云复算、生成网格以及与设计模型对比。

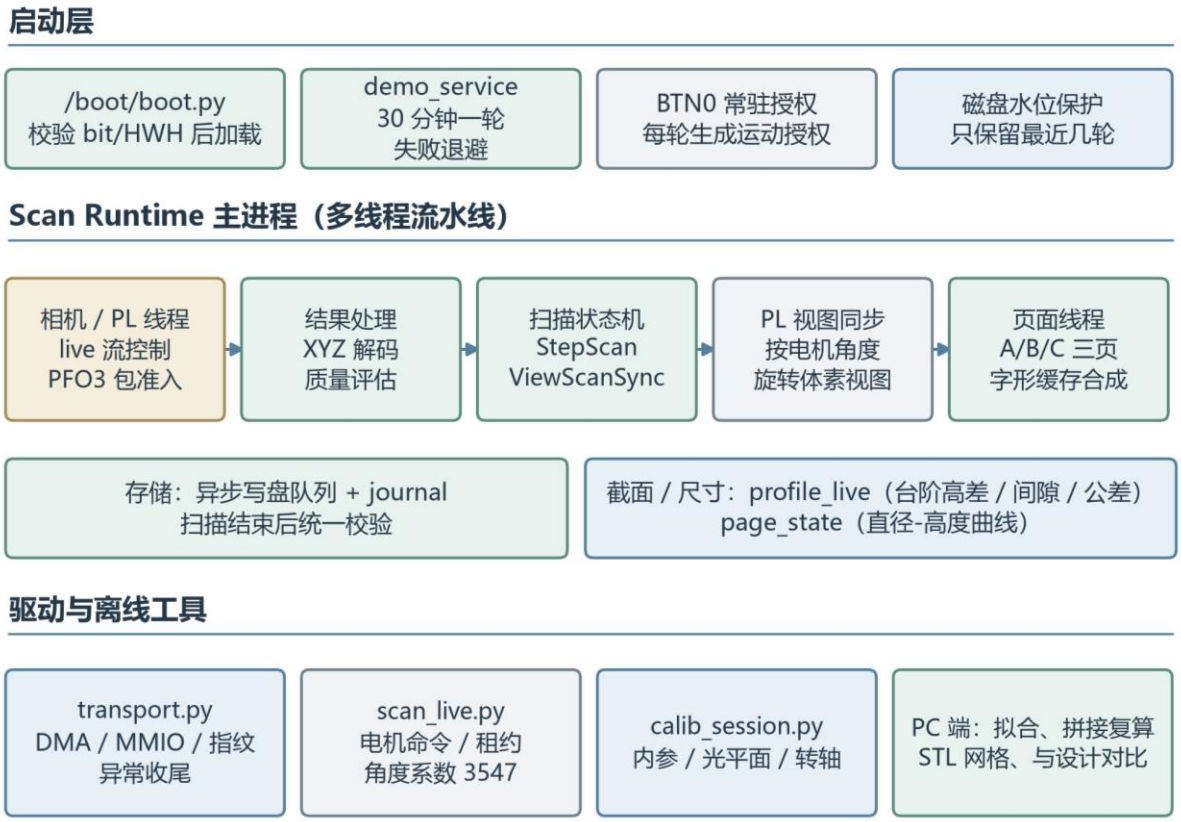


图 11　PS 端软件总体结构

2.3.2 软件各模块介绍

（1）激光条纹中心提取（PL）

原理：对每一行像素先做 3×3 平滑，把响应不低于 24 的连续像素划为一个亮段；在亮段内选曲率最大的点作起点，用该点的灰度梯度 g 和二阶导数矩阵 H（Hessian 矩阵）求出条纹的法线方向 n，再沿法线做一次牛顿迭代，得到亚像素偏移：

$$t = -(n^{T}g)/(n^{T}Hn),\qquad (u, v) = (u_0, v_0) + t\cdot n，\text{要求 } |t\cdot n| \le 0.5 \text{ 像素}$$

硬件实现：流水线分为预处理、选起点与定方向、亚像素定位、跨行关联四级（图 12），按行流式处理，每行最多保留 16 个候选，两路求解并行。条纹中心用带 12 位二进制小数的定点数输出，并附带宽度、饱和、方向等“拒绝原因”标志。相邻行之间按 6 像素距离和宽度差建立关联：连续 3 行以上相连的点记为预览级；连续 8 行以上，且整帧的点数、有效行数、饱和比例和歧义比例都达标的点记为测量级。模块端口见图 13。

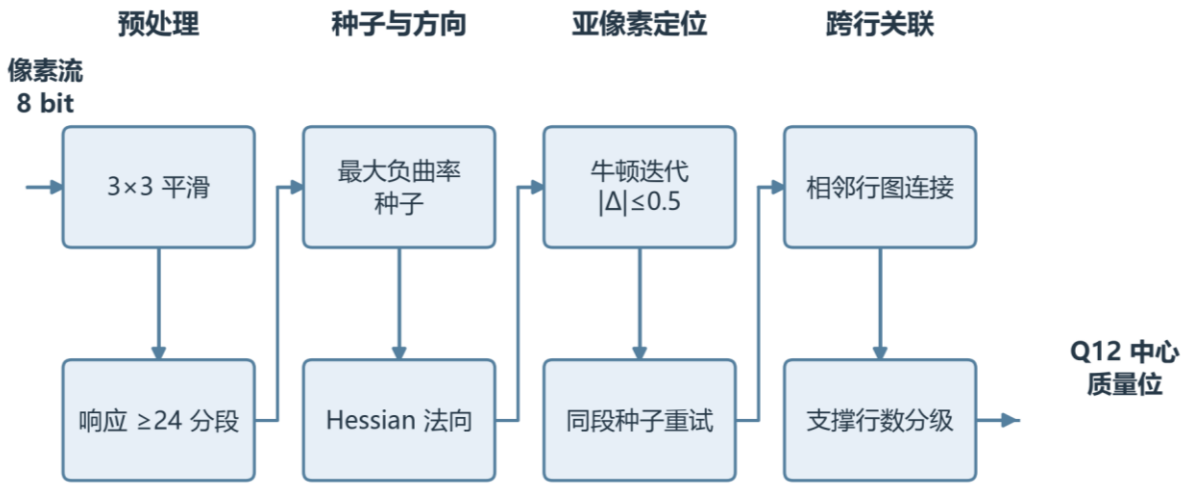


图 12　激光条纹中心提取流水线；Q12 表示坐标含 12 位二进制小数

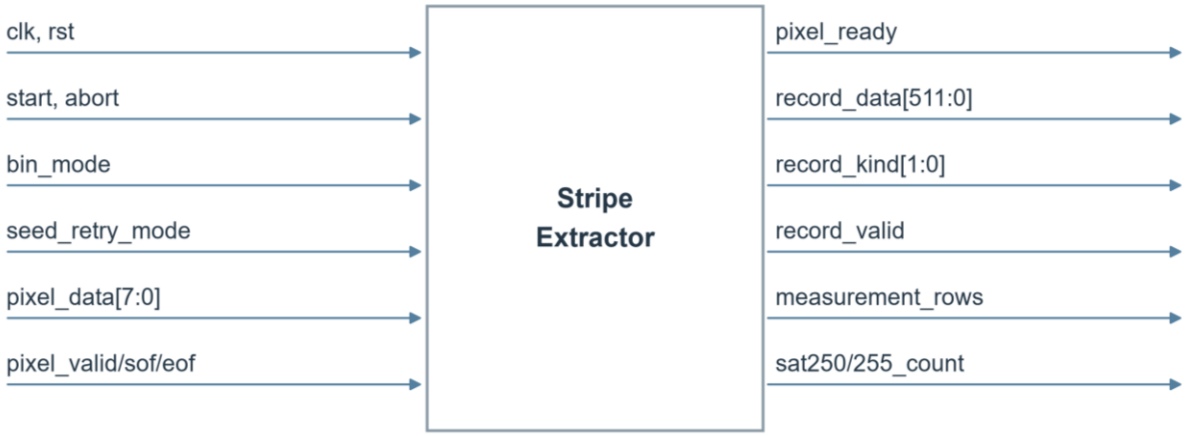


图 13　Stripe Extractor 模块端口

改进：近距离时镜头失焦、激光有散斑，条纹会变宽并碎成许多小段。为此加入“水平 2 合 1”模式（policy 2）：把相邻两个像素取平均再提取中心，窗口内有效点由 21 个增加到 122 个，截面噪声由 4.5 mm 降到 0.63 mm。阶梯圆柱的中、下层点偏少，是因为只试了曲率最大的起点，牛顿迭代一旦失败，整段就被丢弃；于是又加入“同段重试”（policy 3）：失败且这是唯一拒绝原因时，换同一亮段内平滑亮度最大的点再试一次，原来成功的结果保持不变。在 270 帧实拍图像上离线验证，中层每帧有效行数中位数由 9 行增加到 44 行，下层由 9.5 行增加到 28 行。该固件已上板运行，但开机默认使用水平 2 合 1 模式，同段重试需切换到全宽模式才起作用。

上板验证：图 14 是相机实拍帧：激光条纹打在阶梯圆柱上，每一层的条纹都被提取到了。

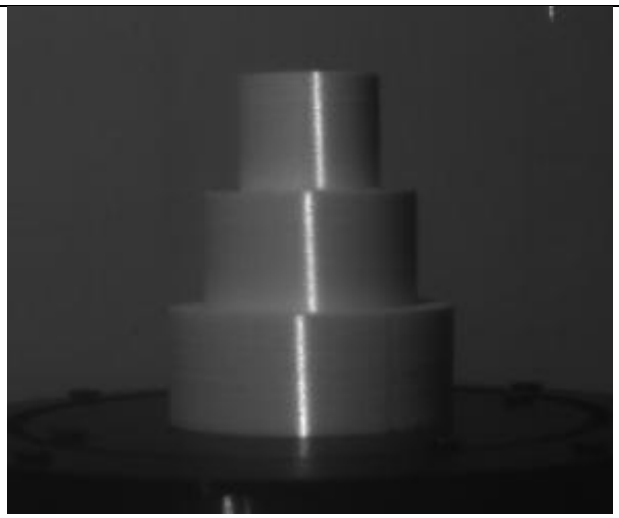


图 14　相机实拍：激光条纹投射在阶梯圆柱上

（2）几何解算（PL）

原理：先按标定好的畸变系数把条纹中心(u, v)校正为理想成像位置，得到一条从相机出发的射线 r = (x, y, 1)。激光平面方程为 n·X + d = 0，射线与激光平面的交点就是条纹点在相机坐标系下的位置：P = λr，λ = −d / (n·r)。转台转过角度 θ 后，再把 P 绕转台轴线(p, a)反向旋转 θ，就回到物体自己的坐标系：

$$Q = R(a, -\theta)(P - p) + p, \qquad \theta = \text{脉冲数} \times 2\pi / 3547$$

硬件实现：乘法用 64 位有符号乘法器（由 DSP 级联），除法用 Q30 移位相减除法器，输出以 1/65536 mm 为单位（Q16）的毫米坐标。参数通过 cfg_*端口写入，按批次（epoch）整体提交，保证一帧之内参数不变（图 15）。在实拍帧上，结果与浮点参考模型逐项一致。

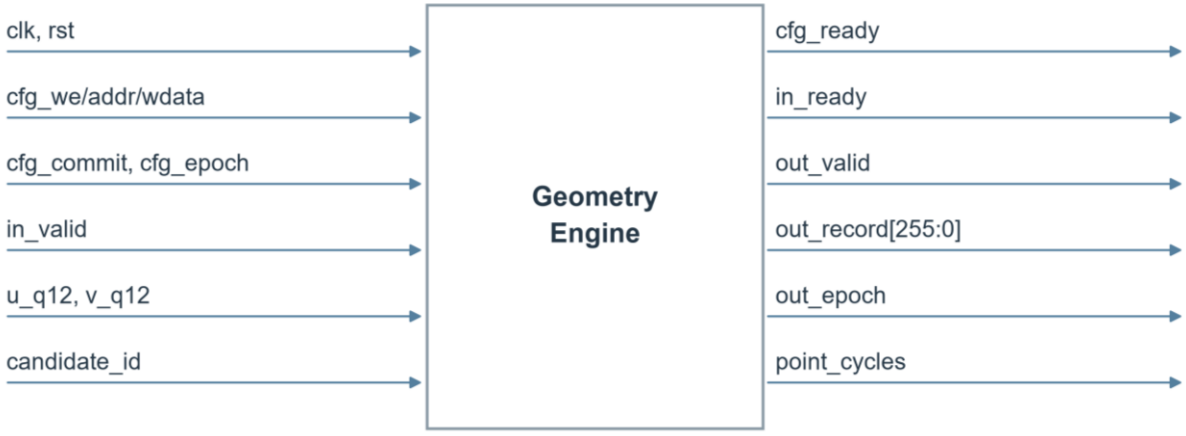


图 15　Geometry Engine 模块端口

（3）旋转扫描状态机（PS）

状态机如图 16 所示。每个视角下发 18 步运动并续租看门狗；运动结束后，等 PL 数据包的序号越过运动结束时刻，确保取到的是转台停稳之后的图像；再检查点数和质量，不合格最多重采 2 次，仍不合格就按预览点接收，保证每个角度都有数据。200 个视角共 3600 步，约转一整圈。再次按下 BTN0 或超时，会立即停止，并关断电机和看门狗输出。

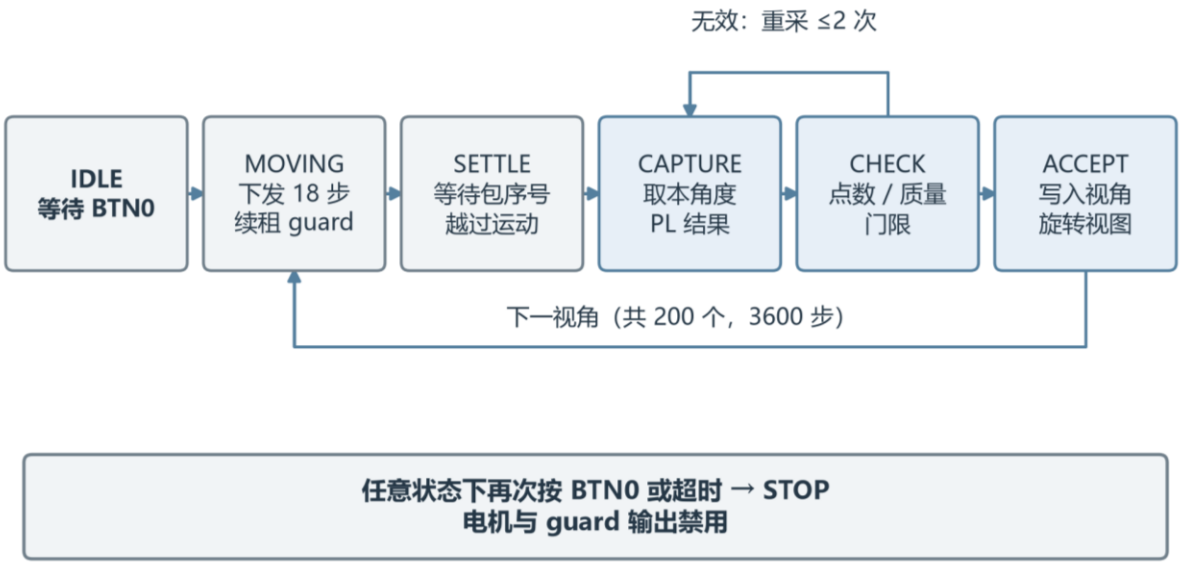
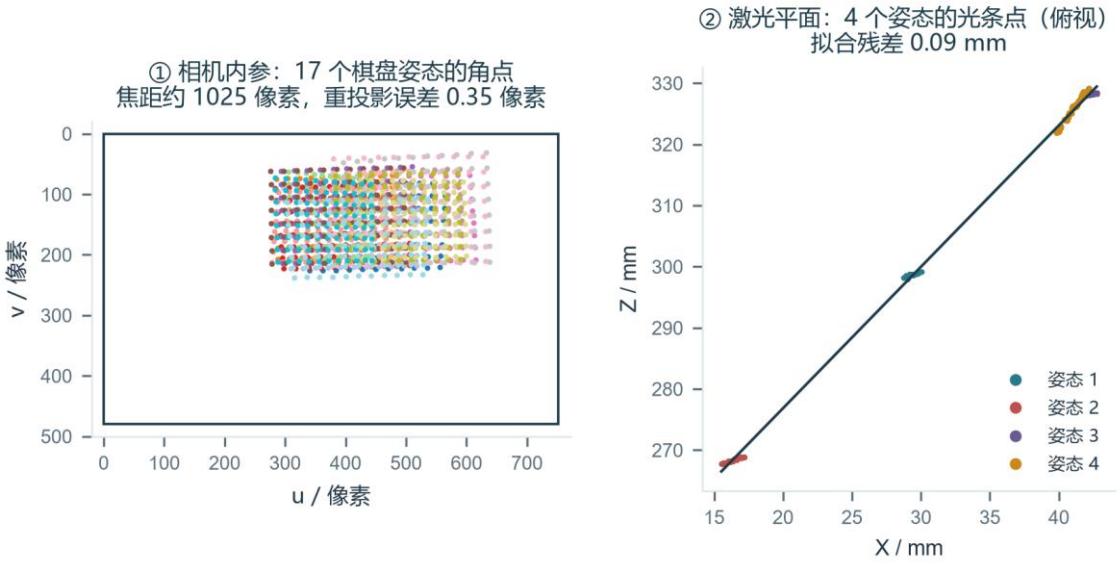


图 16　旋转扫描状态机

（4）标定（PS + PC）

标定分四步（图 17）：①用名义 6 mm 方格的棋盘，在多个姿态下拍照，求相机内参和镜头畸变（标定记录：17 个姿态，焦距约 1025 像素，重投影误差 0.35 像素）；②棋盘靠稳后分别拍关激光和开激光的图像，由棋盘位姿恢复板面上的激光交线，拟合激光平面；③把棋盘放在转台上旋转拍摄，求转台轴线和转角换算系数；④用阶梯圆柱扫一圈，微调激光平面偏移和转轴位置。需要说明的是，第④步用到了阶梯圆柱，所以阶梯圆柱的复测只能当作复核；真正独立的检验用的是花瓶和卡尺核对（见第 3 部分）。本次补充的现场图像和量具核对见图 17a、17b。



③ 转轴：棋盘随转台旋转拍摄 39 个视角；转角系数＝实测 / 指令＝0.902，即 3547 脉冲 / 圈。

④ 阶梯圆柱微调：扫描一圈（200 视角），调整光平面偏移与转轴位置。

⑤ 独立检验：更换样件，用卡尺核对。

图 17　标定流程与标定数据

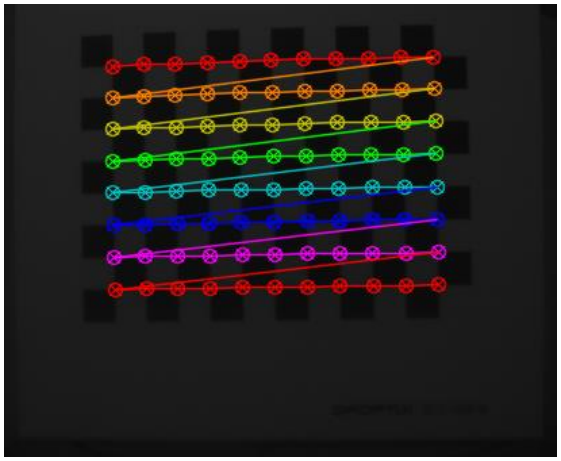
标定补充（一）：标定板的使用

尺度来源：标定用的是贴在硬板上的黑白棋盘，图像中检测到 11×8 个内角点，共 88 个点，格宽按 6 mm 建立空间坐标。这个 6 mm 是沿用以往的名义值，这次没有重新测量格宽。棋盘负责建立像素与空间坐标的对应关系，卡尺负责提供物体的参考尺寸，二者各司其职。

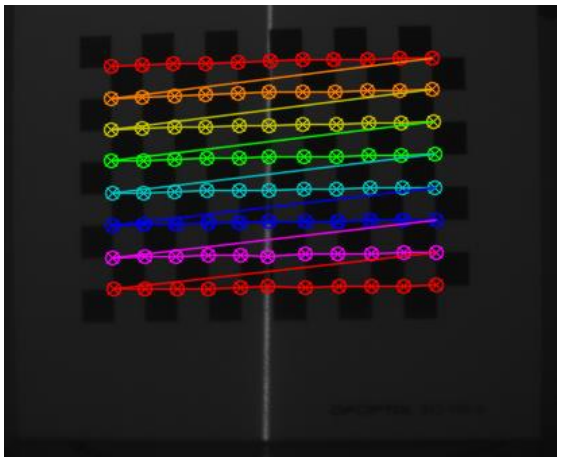
相机与激光：先关掉激光，识别棋盘角点，用多个姿态的图像求内参和畸变；再求激光平面：同一姿态下棋盘保持不动，各拍一张关激光、一张开激光的图像，结合棋盘位姿恢复出板面上的激光线。图 17a 是本次保留的两组现场图像，彩色点线是角点检测的结果。这两个姿态的数据已能拟合出一个激光平面候选，但还没有通过完整的独立检验，也没有作为正式标定装入系统。

转轴与验证：转轴和角度换算要靠旋转观测来确定，只凭一张棋盘或一个圆柱的尺寸，无法同时确定全部参数。台阶块、阶梯圆柱和花瓶的参考尺寸只用来检验，不放进几何拟合——拟合用的数据和检验用的数据必须分开，否则精度结论不可信。另外，相机的重投影残差也不能直接当作整机的毫米误差。

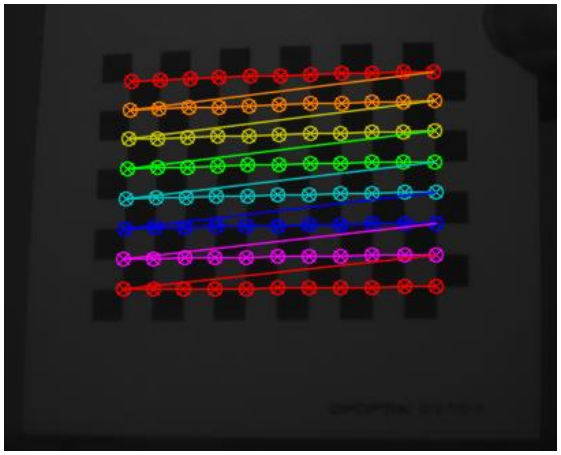
姿态 1 · 激光关 · 帧 40



姿态 1 · 激光开 · 帧 185



姿态 2 · 激光关 · 帧 690



姿态 2 · 激光开 · 帧 595

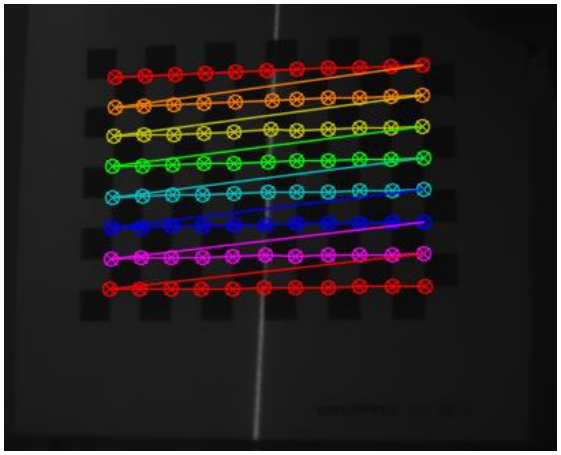


图 17a　棋盘现场图像：姿态 1 关/开激光（帧 40/185），姿态 2 关/开激光（帧 690/595）

标定补充（二）：卡尺核对与三件展示样件

尺寸核对：使用游标卡尺测量台阶块相邻台面的高差、阶梯圆柱各段外径和花瓶颈部外径。台阶块相邻高差为 2、3、5 mm，阶梯圆柱下、中、上段外径为 40、30、20 mm，花瓶颈部为 16.5 mm。这些实测尺寸用于核对扫描结果，不参与光平面、转轴和形状的拟合。

怎么判断对不对：标定固定之后，用扫描值减去卡尺参考值，分别列出偏差、多次测量的离散程度，以及无法测量的项目。台阶块要先确认测量方向和台面基准，斜截面上的偏移不能直接当作台阶高度；阶梯圆柱的设计段高（16 mm、总高 48 mm）和卡尺读数要分开写；花瓶的设计模型只用来对比外形，不能代替实物的卡尺读数。

图 17b 展示台阶块的激光截面，各级高度差为 2、3、5 mm。

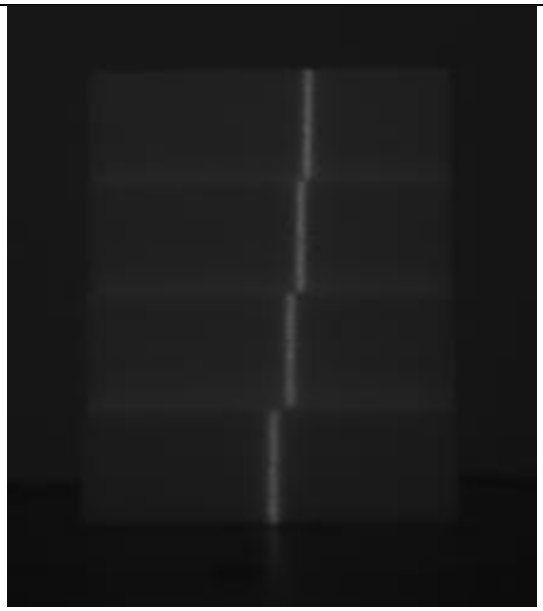


图 17b　台阶块激光截面实拍（帧 1010）；相邻台阶高度差为 2、3、5 mm。

（5）尺寸测量与质量评估（PL + PS）

定点轮廓测量：轮廓统计对每个提交的数据包统计 Z 最小/最大/中位数和 32 层半径带（图 18）；截面测量对激光平面内的截面点排序、分段并拟合直线，输出台阶高差、间隙和噪声（图 19），统计全部在 PL 内完成，PS 只读寄存器。

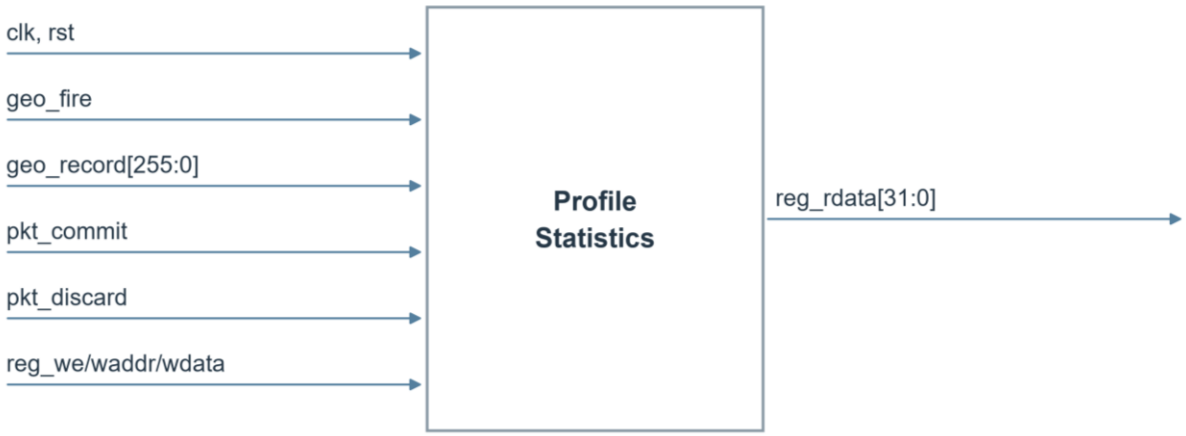


图 18　Profile Statistics 模块端口

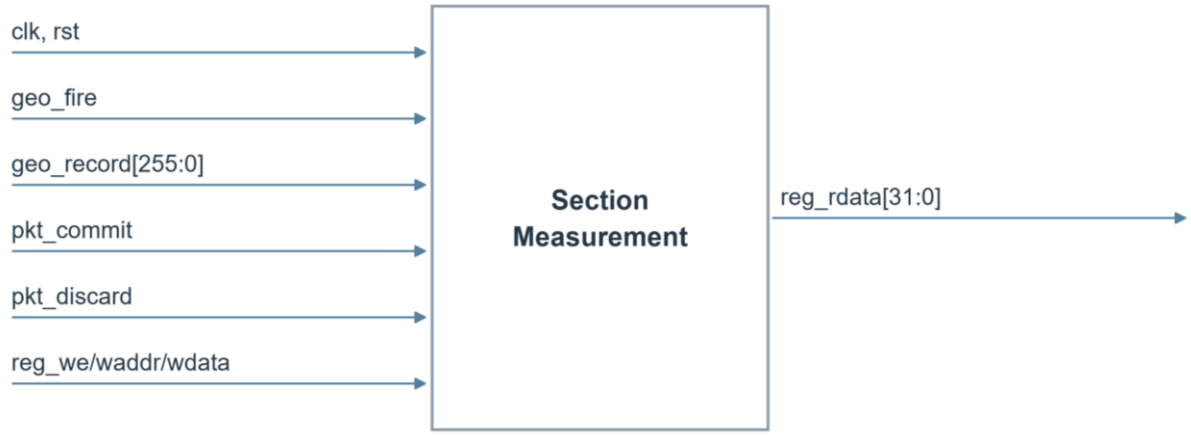


图 19　Section Measurement 模块端口

扫描尺寸判定：扫描完成后，物体坐标点按高度每 1 mm 分层，每层用最小二乘拟合圆得到直径、圆心和圆度，生成直径—高度曲线；按台阶处半径跳变分段，统计各段直径与段高并与公差比较，给出合格/超差。

质量评估与自动曝光：质量评估综合考虑条纹峰值、宽度、饱和像素比例和有效行数，分为“良好/一般/差”三级，差的视角会触发重采；自动曝光根据每帧的峰值和饱和情况，经 SCCB 调节相机曝光行数（图 20），实测 13 s 内由 240 行调到 340 行，且没有饱和。

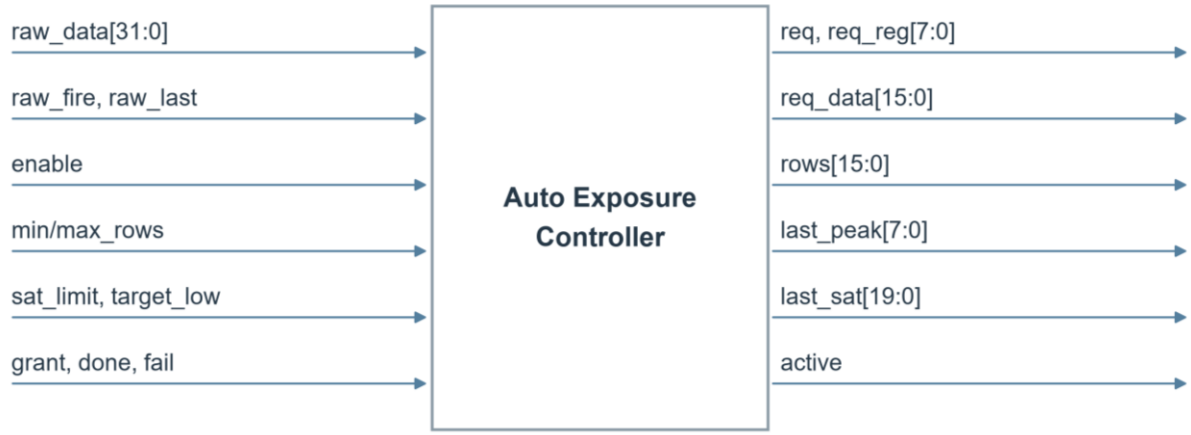


图 20　Auto Exposure Controller 模块端口

（6）点云显示与界面（PL + PS）

点云渲染截取几何点写入 64×64×96 体素，按当前视角投影渲染到双缓冲颜色 BRAM，再在视频时钟域叠加到 HDMI 画面（图 21）。界面分三页，用 BTN1～BTN3 切换，实拍画面见第 3 部分图 24～图 26：A 页为相机画面、条纹中心和 PL 点云；B 页为截面轮廓、台阶高度、间隙与采集质量；C 页为扫描进度、点数和直径—高度曲线。中文字形预先栅格化并缓存，每帧只重绘变化区域。

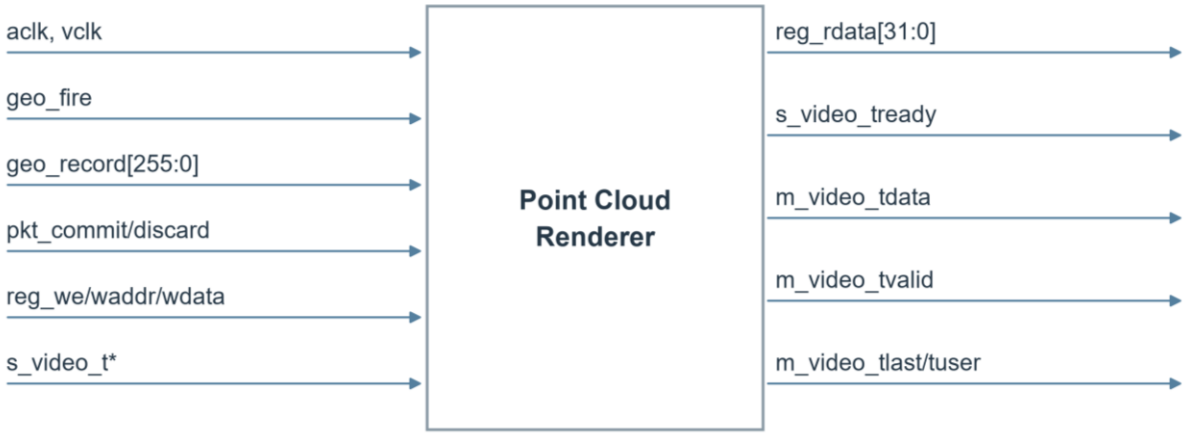


图 21　Point Cloud Renderer 模块端口

2.3.3 开发中的关键问题与解决

表 4　关键问题与解决方法

问题	原因（实测定位）	解决方法与效果
条纹被拉宽、碎段多	近距离失焦与激光散斑	2×水平合并模式：有效点 21→122，噪声 4.5→0.63 mm
自动曝光使条纹饱和	整幅亮度驱动曝光	PL 按条纹峰值与饱和自动曝光，0 饱和
中、下层条纹点少	牛顿迭代只试最大曲率种子	同段种子重试，中层行数 9→44
电机一圈角度不足	实际转角为指令的 0.902	棋盘实测换算为 3547 脉冲/圈
ARM 渲染点云占用高	Python 逐点投影	点云渲染移入 PL，ARM 侧 22→3.3 ms
跨时钟时序违例	约束匹配旧模块名失效	修正约束并签核，CDC 严重项 0
电机失控风险	软件异常时脉冲持续	PL 租约看门狗，停止续租即禁用输出

第三部分　完成情况及性能参数

3.1 整体介绍

系统已完成整机搭建和开机固化：上电后自动加载固件并进入实时页，按 BTN0 完成一次 360°扫描，扫描结果在 HDMI 上实时显示，并可导出点云和网格。整机实物如图 22 所示：左侧为 PYNQ-Z2 和相机，右侧为线激光器和转台，上方显示器正在显示花瓶的实时扫描。

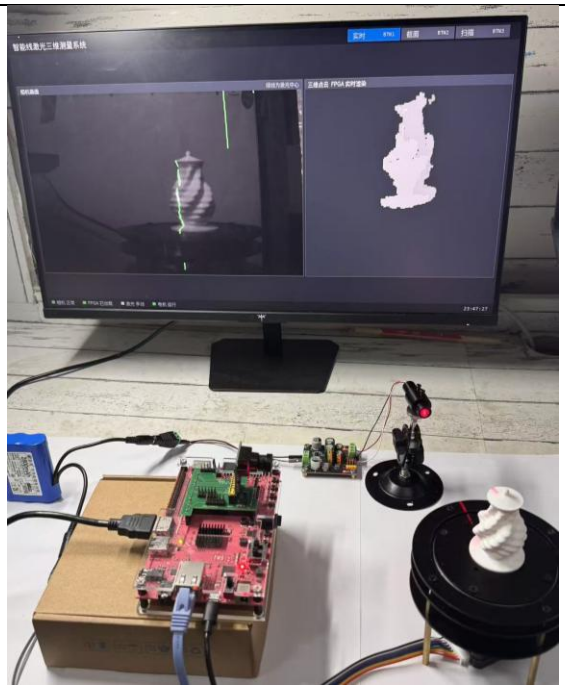


图 22　整机实物与花瓶扫描现场

3.2 工程成果

3.2.1 机械成果

机械部分的实物布局见图 6 和图 22。相机经转接板直接插在开发板上，与开发板一起放在支撑盒上；线激光器装在可调万向支架上，从侧面把光平面打到转台中心；转台由步进电机驱动，样件放在转台中心。三者在标定后保持固定，扫描时只有转台转动。

3.2.2 电路成果

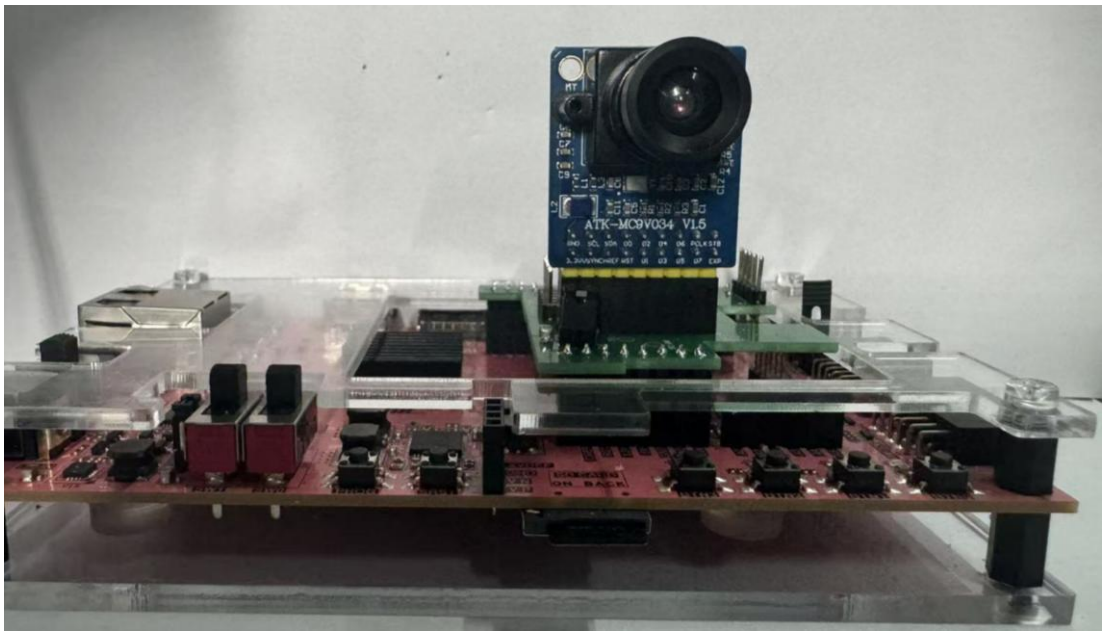


图 23　PYNQ-Z2、相机转接板与 MT9V034 连接实物

3.2.3 软件成果

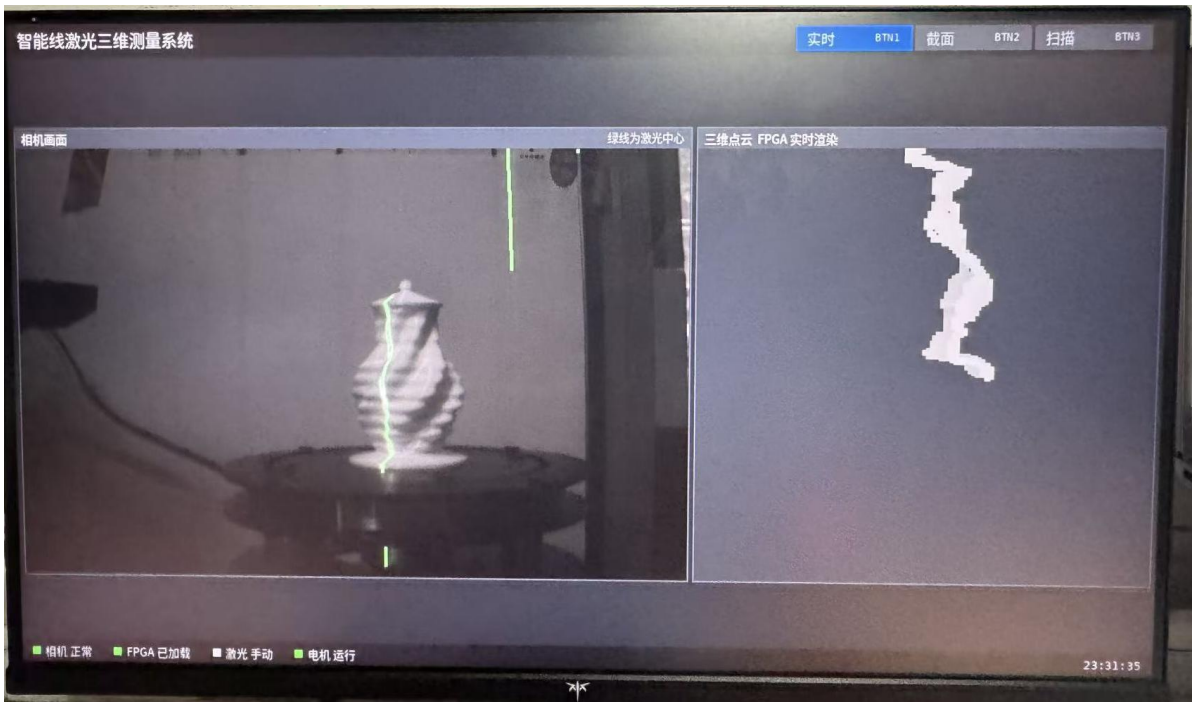


图 24　HDMI 实时页（A 页）：相机画面、条纹中心与 PL 点云

图 24 中，左侧相机画面里的绿色竖线是 FPGA 提取出的条纹中心，右侧是 FPGA 直接渲染的花瓶点云。



图 25　HDMI 截面页（B 页）：轮廓、测量状态与质量提示

图 25 拍摄时转台上放的是花瓶，不是台阶块，因此截面页显示的 191.73 mm 带有“不稳定”标记，这里只用来展示界面，不作为测量结果。

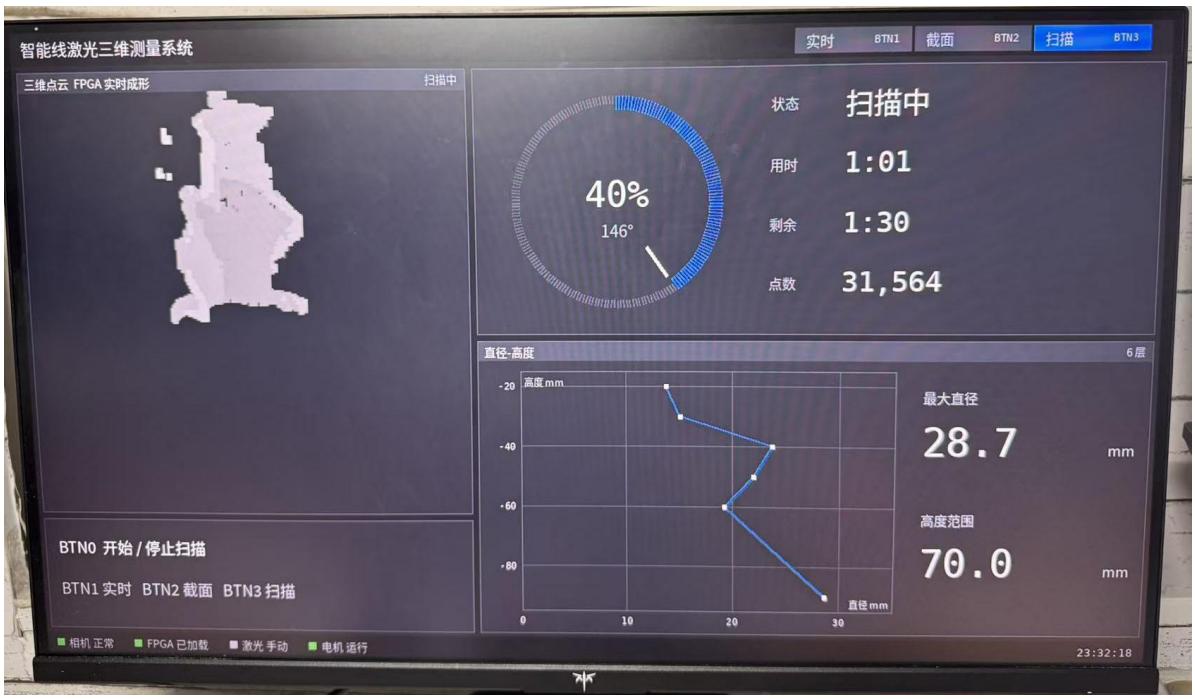


图 26　HDMI 扫描页（C 页）：扫描进度与点云生成

图 26 是扫描进行中的画面：转台转到 146°（40%）时用时 1 分 01 秒，预计还需 1 分 30 秒，已累积 31,564 个点；左侧点云随转台同步旋转，右下的直径—高度曲线随扫描逐层生成。

3.3 特性成果

（1）功能完成情况

表 5　功能完成情况

功能	完成情况	验证方式
360°旋转扫描	完成，BTN0 一键启动	多次整圈实扫（阶梯圆柱、花瓶等）
多截面尺寸测量与判定	完成	阶梯圆柱与花瓶颈部卡尺对比
定点轮廓实时测量	完成，PL 截面模块	台阶样件实时测量
质量评估与重采	完成	每圈测量级视角 195～199 个，跳过 0 个
自适应曝光	完成，PL 自动曝光	曝光 240→340 行，0 次写失败
PL 点云实时显示	完成	体素视图读回与模型逐像素一致
开机固化	完成	断电重启后自动进入演示

（2）处理速度

同一条纹提取算法在不同平台上的单帧耗时见表 6。PL 版本以流水方式随像素到达处理，跟上相机帧率且耗时固定；PC 端 Python 参考实现与 PL 逐位一致，用作正确性基准。

表 6　条纹提取处理速度对比

实现	平台	单帧耗时	说明
PL 流水线（本作品）	XC7Z020，100 MHz	约 37.7 ms/帧（与相机帧周期相同）	含几何解算，逐帧处理
Python 参考模型	PC，Intel 第 13 代	约 137 ms	同一算法，含条纹关联，不含几何解算
PS 早期算法	ARM Cortex-A9	约 1.21 s	早期 Python 实现，仅供参考

（3）资源与功耗

表 7　FPGA 资源占用与功耗（实现后）

资源	使用量	总量	占用率
LUT	39,111	53,200	73.5%
FF	50,902	106,400	47.8%
BRAM	110.5	140	78.9%
DSP	134	220	60.9%
片上功耗 / 结温	2.37 W	—	52.3 ℃
时序	WNS +0.230 ns	WHS +0.011 ns	CDC 严重项 0

（4）中心提取精度

真实图像没有逐点真值，因此用已知中心线的合成条纹评价（斜线、正弦、宽度变化、背景梯度和噪声，开发集 16 例、留出集 16 例，参数固定后才运行留出集）。误差为提取中心到解析中心线的法向距离（表 8）。

表 8　合成真值条纹上的中心提取误差（像素）

数据集	中位数（各例 P50）	中位数（各例 P95）	>0.5 像素点数
开发集（16 例）	0.031	0.060	2
留出集（16 例）	0.023	0.053	0
留出集（种子重试）	0.025	0.056	0

（5）尺寸精度

表 9 给出了扫描与卡尺的对照结果。阶梯圆柱参与过标定微调，所以换位重扫只能说明该样件复核通过，不能算独立精度验证；花瓶是独立打印件，颈部卡尺读数 16.5 mm。量具核对结果为：台阶块相邻高差 2、3、5 mm，阶梯圆柱下、中、上段直径 40、30、20 mm。光路调整后标定已经更新，因此上表的误差范围不直接当作当前精度。

表 9　扫描与卡尺尺寸对照（阶梯圆柱参与过参数调整）

物体	项目	卡尺/mm	扫描/mm	误差/mm
阶梯圆柱	大段直径	40.0	39.68	−0.32
	中段直径	30.0	29.90	−0.10
	小段直径	20.0	19.88	−0.12
	中、小段段高	16.0 / 16.0	16.0 / 16.0	0
波纹花瓶	颈部最小直径	16.5	16.23～16.55	−0.27～+0.05

（6）扫描结果

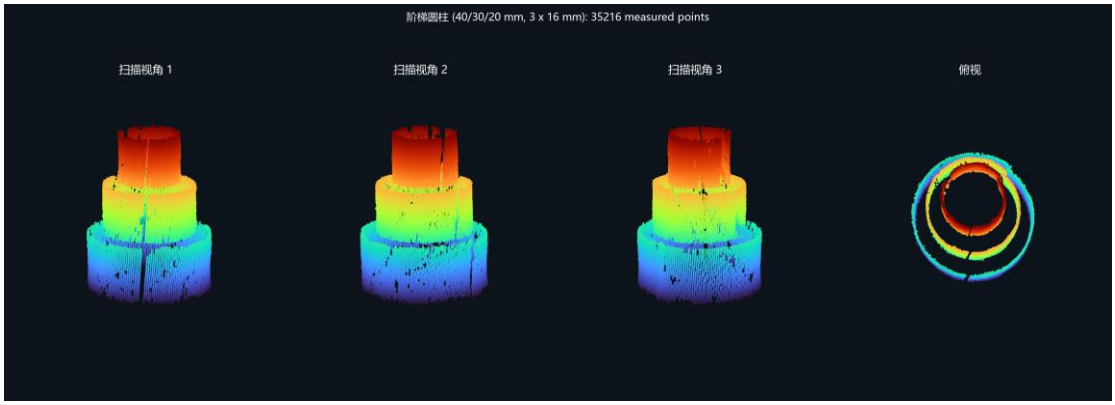


图 27　阶梯圆柱扫描点云（颜色表示高度，右为俯视）

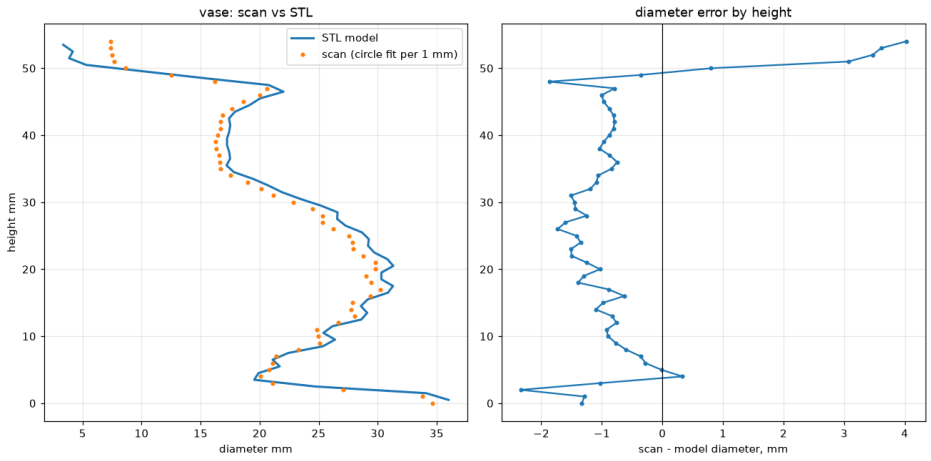


图 28　花瓶扫描与 STL 设计：各高度直径对比



图 29　由一次扫描生成的封闭网格（左起：花瓶两侧、花瓶设计模型、阶梯圆柱）

（7）与同类方案对比

表 10　与同类线激光 FPGA 方案对比

方案	平台	在 FPGA 内完成	三维/测量	精度
Wang 等[7]	XC7Z020 + CMV4000	中心提取	PC 端完成	0.15 mm（量块）
Lin 与 Tey[15]	FPGA 实验箱 + NTSC 相机	二值化、骨架、重建、渲染	FPGA 内，无定标精度	未报告
本作品	XC7Z020 + MT9V034	中心、几何、统计、截面、渲染	板上完成，无需 PC	0.10～0.32 mm（卡尺，阶梯圆柱为复核）

与 Wang 等的方案相比，本作品帧率较低，但三维解算、尺寸测量和显示全部在板上完成；与 Lin 和 Tey 的方案相比，本作品给出了亚像素中心、完整标定链和以卡尺为真值的尺寸精度。

第四部分　总结

4.1 可扩展之处

一是转轴标定：当前转轴由回转对称样件精修，对倒置摆放和非对称物体，误差会随高度增大，计划用非对称标定块联合求解转轴与转角。二是扫描速度：PL 已能处理每一帧，可以把点云拼接也移入 PL，实现连续旋转扫描。三是测量范围：增加第二个相机或激光来消除遮挡，用多次曝光应对高反光表面。四是测量功能：加入公差模板和 CAD 比对，自动生成检测报告。

4.2 心得体会

这个作品让我们真正体会到，三维测量不是“把点画出来”那么简单，而是光学、电路、数字逻辑、标定和机械几个方面同时成立的结果。最初相机已经能出图，条纹也能画在屏幕上，但量出来的尺寸偏差达到十几个百分点。逐级排查后才发现，误差来自好几个层面：镜头没有对焦在工作距离上，条纹被拉宽成散斑，算法在同一行找到多个候选；电机标称 3200 步一圈，实测却要 3547 步；标定得到的激光平面在标定姿态附近很准，到了物体所在的位置就有偏差。每一个问题都要回到同一帧原始图像上复查，才能判断该改光路、算法还是标定。为此我们从一开始就坚持把原图、参数、中间结果和请求编号一起保存，这让很多说不清的“玄学”问题变成了可以复现的问题。

在 FPGA 设计上，我们先用 Python 写参考模型，再写 RTL，仿真时逐字段比对，上板后再用实拍帧比对，这套方法帮我们发现了好几处位宽和舍入错误。时序收敛是另一个难点：把几何解算、测量统计和点云渲染都放进 PL 后，资源用到七八成，多次出现跨时钟域约束漏配、LUTRAM 读路径过长的问题。我们学会了先看报告里的最差路径，再决定是插寄存器、改存储类型还是调整约束，而不是盲目重跑。

开发过程中我们大量使用了大模型辅助：让它阅读 RTL 和报告、生成测试和板上脚本、分析失败日志，并把常用的 FPGA 调试流程整理成可复用的技能包。我们的体会是，大模型能显著加快编码和排查，但结论必须由实测证据支撑：每次加载固件、转动电机都由人确认，每个精度数字都要对应到卡尺读数和原始数据。正是这种“证据优先”的协作方式，让我们在有限时间内把系统从静态截面推进到了能一键扫描、有完整标定与验证流程的作品。

第五部分　参考文献

[1] ZHANG Z. A flexible new technique for camera calibration[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2000, 22(11): 1330-1334.

[2] STEGER C. An unbiased detector of curvilinear structures[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1998, 20(2): 113-125.

[3] FISHER R B, NAIDU D K. A comparison of algorithms for subpixel peak detection[M]//SANZ J L C. Image Technology. Berlin: Springer, 1996: 385-404.

[4] BLAIS F. Review of 20 years of range sensor development[J]. Journal of Electronic Imaging, 2004, 13(1): 231-240.

[5] ZHOU F, ZHANG G. Complete calibration of a structured light stripe vision sensor through planar target of unknown orientations[J]. Image and Vision Computing, 2005, 23(1): 59-67.

[6] ATIF M, LEE S. FPGA based adaptive rate and manifold pattern projection for structured light 3D camera system[J]. Sensors, 2018, 18(4): 1139.

[7] WANG J, JIN L, LI J, et al. High-speed laser centerline extraction and heterogeneous measurement architectures for low-cost hardware[EB/OL]. https://ssrn.com/abstract=4588156.

[8] GU Q, HERAKLEOUS K, POULLIS C. A structured-light scanning software for rapid geometry acquisition[EB/OL]. https://arxiv.org/abs/1406.6595.

[9] GRANS S, TINGELSTAD L. Blazer: laser scanning simulation using physically based rendering[EB/OL]. (2021) https://arxiv.org/abs/2104.05430.

[10] BRADSKI G. The OpenCV library[J]. Dr. Dobb's Journal of Software Tools, 2000, 25(11): 120-125.

[11] Xilinx. Zynq-7000 SoC technical reference manual (UG585)[EB/OL]. https://docs.amd.com.

[12] Xilinx. AXI DMA LogiCORE IP product guide (PG021)[EB/OL]. https://docs.amd.com.

[13] Xilinx. Vivado design suite user guide: design analysis and closure techniques (UG906)[EB/OL]. https://docs.amd.com.

[14] Xilinx. PYNQ: Python productivity for Zynq[EB/OL]. https://pynq.readthedocs.io.

[15] LIN J, TEY E. 3D scanner[R]. Cambridge, MA: MIT 6.111 Final Project, 2016.

[16] ON Semiconductor. MT9V034 1/3-inch wide-VGA CMOS digital image sensor datasheet[EB/OL].

[17] 正点原子. PD42S1 步进电机闭环驱动器用户手册[EB/OL]. http://www.alientek.com.

附录一 优化与资源对照

本附录汇总正文中的实测优化结果，并补充同器件、同工具版本下的功能扩展资源对照。所有测量数值沿用原记录。

优化项目	优化前	优化后
窗口内有效点数	21	122
截面噪声	4.5 mm	0.63 mm
中层每帧有效行数中位数	9 行	44 行
ARM 投影耗时	约 22 ms	约 3.3 ms

前三项对应条纹提取策略的对照，投影对应软件批处理优化，不能合成为同一算法的统一加速比。

功能扩展前后整板资源对照

资源与时序	双方向基础通路	完整测量显示通路
LUT	32,094 / 60.33%	39,111 / 73.52%
FF	43,185 / 40.59%	50,902 / 47.84%
BRAM 等效块数	108.5 / 77.50%	110.5 / 78.93%
DSP	64 / 29.09%	134 / 60.91%
WNS / WHS	+0.150 / +0.008 ns	+0.230 / +0.011 ns

两版均为 XC7Z020、Vivado 2025.2 的布线后结果，系统时钟 100 MHz、视频逻辑 125 MHz。完整候选新增截面测量并调整方向重试，资源增量用于功能扩展，不作为同功能纯提速。截面测量层级实际使用 70 个 DSP、2 块 BRAM。原始报告见源码包 build/evidence/reports；基础通路报告见 build/evidence/baseline_reports。

附录二 从零复现与验证

安装 Python 3.10 或更高版本、Vivado 2025.2 及 XC7Z020 器件支持；RTL 行为测试另需 Icarus Verilog。解压代码 RAR 后进入 project 根目录，在新的输出目录运行以下命令。

1 检查交付文件与仓库结构

```
python -B tools/check_repository.py
python -B skill/verify_bundle.py .
```

2 恢复原始路径并运行软件回归

```
python -B build/restore_sources.py --output ../work
python -m pip install -r sim/requirements_pc.txt
python -B sim/run_runtime_tests.py --work ../work --output ../results
```

3 运行最小 RTL 集成验证

```
python -B sim/run_rtl_integration.py --work ../work
  --output ../rtl_results --iverilog iverilog --vvp vvp --cases empty
```

4 从源码完整构建

```
python -B build/reproduce.py --work ../build_work
  --output ../build_out --vivado vivado
```

多行命令需合并为一行执行，输出目录必须不存在。构建器恢复原始文件名后调用完整工程脚本，不以旧检查点冒充源码构建。详细说明见 docs/build_and_deploy.md。

文件检查、软件回归、RTL、整板实现和实物测量分别验收。本次已修复运行测试、旧格式严格读回及稳态图像质量统计路径，并完成离线回归和软件固件配套检查。原固件和构建记录保留，当前软件另立发布配套记录；完整结果与复现命令见 report/release_validation.json。

附录三 可复用技能与协作记录

技能包提炼自完整归档和固件软件配套核对。Bundle Checker 检查交付文件集合，Overlay Release Checker 检查 bit、HWH 和受保护运行软件。工具仅依赖 Python 标准库，离线只读，可用于其他 PYNQ 工程。

运行与验证

```
python -B skill/verify_bundle.py .
python -B skill/test_pynq_release_check.py
```

发布工具通过文件损坏、路径、协议身份和容量等故障检查；新增配套测试核对原始 PL 编译输入、寄存器接口及 ARM 原生库架构，截面 RTL 与定点模型逐字段一致。原始失败证据保留。检查结果、适用范围和开发来源见 report/release_validation.json、skill/readme.md 与 report/skill_development.md。

实际协作节选

案例一　工程整理。用户提示："你把东西全完善一下。我一整个仓库先把我完善一下。"模型回答："我会先检查现有仓库和提交包，再参考成熟 FPGA 项目的文档组织方式"。复核发现归档内容和软件绑定是不同问题，分别形成文件集合与配套组合检查；Git 元数据另行处理，额外业务文件仍须检查。

案例二　界面展示。用户提示："我说的不是自己拍的,而是就是之前生成的那种图片。"模型回答："你要的是之前程序直接生成的界面图，不是手机拍显示器的照片。"据此找回实时、截面和扫描三页的原始输出，并统一语言入口；界面数值及测量算法保持不变。

以上为真实交互短节选，完整案例、纠错过程及对应文件见 report/ai_collaboration_records.md。技能与协作材料保留在源码包内，不再另装重复压缩包。